Project Proposal

Real-Time Intelligent Medical-Surgical Sales Platform

1. Executive Summary

This project proposes the design and development of a multi-role, real-time healthcare sales intelligence platform for medical-surgical product sales. The platform will support three primary user groups: administrators, sales representatives, and customers. The system will capture transactional and behavioral data in real time, generate invoices, process payments through Stripe in test mode, maintain rich customer profiles, and provide product recommendations using multiple machine learning modules.

The platform is designed to solve a core business challenge in medical-surgical sales environments: sales representatives often respond only to the subset of products customers explicitly request, which can lead to missed cross-sell and upsell opportunities. By combining customer profiling, purchase behavior analysis, real-time tracking, and machine learning driven recommendations, the system will help sales representatives proactively recommend relevant products and improve sales outcomes.

The solution will be implemented using FastAPI for backend services, React for the frontend, Python for machine learning, Apache Kafka for event streaming, and WebSockets for real-time client updates. PostgreSQL will be used as the primary operational database due to its reliability, relational consistency, analytical friendliness, and strong free open source ecosystem.

2. Project Objectives

The main objectives of the project are as follows:

1. Build a multi-role web platform for admin users, sales representatives, and customers.
2. Capture orders, invoices, customer interactions, and wishlist behavior in real time.
3. Generate product recommendations using multiple machine learning modules.
4. Provide live analytics dashboards for administrators using Kafka and WebSockets.
5. Maintain customer profiles with historical purchases, preferences, and behavioral signals.
6. Enable sales representatives to access customer-specific intelligence before and during sales interactions.
7. Simulate realistic payment and invoice workflows through Stripe test mode.
8. Deliver a production-style architecture that demonstrates both business understanding and technical depth.

3. Proposed User Roles and Responsibilities

3.1 Administrator

The administrator will have access to platform-wide monitoring, analytics, and management functions.

Admin capabilities

- View total sales and sales trends in real time
- Track product performance and revenue contribution
- Monitor sales representative performance
- View customer segment level and cluster level insights
- Manage product catalog, categories, and pricing
- Monitor recommendation usage and conversion
- Review wishlist trends and demand signals
- View invoice and payment status across the platform
- Manage user roles and permissions

3.2 Sales Representative

The sales representative will use the platform to understand customer needs, view recommendations, and complete transactions.

Sales representative capabilities

- Search and access customer accounts
- View customer profile and purchase history
- See recommended products based on historical data and behavioral patterns
- View customer wishlist activity
- Generate quotes and invoices
- Place orders and initiate payment workflow
- Receive real-time alerts or insights on customer activity
- View reorder and cross-sell opportunities

3.3 Customer

The customer portal will act as a business-focused e-commerce interface.

Customer capabilities

- Browse and search products
- View order history and invoice history
- Add products to wishlist
- Add products to cart and place orders

- View personalized product recommendations
- See pricing, taxes, and final payable amount
- View estimated insurance-adjusted payable amount if included as a rules-based estimator
- Manage profile details

4. Functional Scope and Features

4.1 Core Platform Features

Authentication and Authorization

- Role-based login system
- Secure user registration and login
- JWT based authentication and route protection
- Refresh token based session continuity
- Protected APIs and role-based access control for admin, sales representative, and customer access

Product Catalog Management

- Product list with metadata
- Product categories and subcategories
- Search and filter functionality
- Price and availability display
- Product detail view

Customer Profile Management

- Customer account details
- Segment and cluster assignment
- Historical purchase summary
- Order frequency and average spending
- Wishlist activity
- Recommended items and recommendation history
- Behavioral summary including recently viewed and frequently purchased categories

Order and Invoice Management

- Add to cart workflow
- Order placement
- Stripe checkout integration in test mode
- Invoice generation

- Tax calculation
- Order status tracking
- Payment status tracking

Recommendation System Features

- Product recommendations for customers
- Product recommendations for sales representatives to pitch
- Similar customer based recommendations
- Frequently bought together suggestions
- Cluster based missing product suggestions
- Reorder reminders and next likely purchase category indicators

Real-Time Features

- Live sales dashboard updates
- Live product trend tracking
- Live payment and order status updates
- Live wishlist and customer behavior updates where appropriate
- Live alerts for sales representatives and administrators

Analytics and Reporting

- Revenue by date, category, product, and sales representative
- Top products and top categories
- Customer cluster analytics
- Recommendation conversion analytics
- Wishlist to purchase conversion tracking
- Sales representative performance tracking
- Average basket size and order volume trends

5. Recommended Technology Stack

5.1 Frontend

Framework: React

Supporting libraries and tools

- React Router for routing
- Axios for API calls
- Zustand or Redux Toolkit for state management
- Tailwind CSS or Material UI for interface styling

- Recharts or Chart.js for dashboard visualizations
- WebSocket client for live updates

Why React

- Strong component-based design
- Well suited for role-based dashboards
- Good support for real-time UI updates
- Fast development cycle for a full-time 22-day project

5.2 Backend

Framework: FastAPI

Supporting libraries and tools

- Pydantic for request and response validation
- SQLAlchemy for ORM
- Alembic for schema migrations
- FastAPI JWT Auth or python-jose for token handling
- Uvicorn for serving
- Celery or background tasks only if necessary

Why FastAPI

- Fast development speed
- Strong Python ecosystem compatibility
- Excellent for API-first architecture
- Native async support for real-time and streaming oriented workloads
- Clean integration with ML inference endpoints

5.3 Machine Learning Layer

Language: Python

Primary libraries

- pandas for data manipulation
- numpy for numerical processing
- scikit-learn for clustering and baseline models
- XGBoost or LightGBM for predictive ranking and propensity modeling
- mlxtend for association rule mining
- implicit or Surprise for collaborative filtering if needed
- joblib or pickle for model serialization

- Optuna if hyperparameter tuning time permits

5.4 Event Streaming

Platform: Apache Kafka

Why Kafka

- Enables event-driven architecture
- Supports real-time sales and user behavior event processing
- Decouples operational systems from analytics and notification systems
- Demonstrates strong engineering depth for a capstone project

5.5 Real-Time UI Delivery

Protocol: WebSockets

Why WebSockets

- Pushes live updates from server to client without repeated polling
- Suitable for dashboards, notifications, and order status updates
- Works well with Kafka consumers that push processed events to clients

5.6 Payment and Invoice

Provider: Stripe test mode

Why Stripe

- Free for testing and development
- Provides realistic checkout flow
- Supports test cards for demos
- Helps simulate an end-to-end transactional workflow

5.7 Database Recommendation

Primary database: PostgreSQL

Why PostgreSQL is the recommended choice

- Free and open source
- Strong relational consistency for transactional data

- Excellent support for joins across users, products, orders, invoices, and recommendations
- Better fit than NoSQL for invoice, order, and reporting data
- Supports JSON columns if some semi-structured storage is needed
- Integrates well with analytics workflows and SQL-based aggregation

NoSQL recommendation

A NoSQL database is not necessary for the core project. PostgreSQL alone is sufficient for the MVP and likely the full capstone scope.

If an additional NoSQL option is desired later, Redis can be added for caching and fast session or notification handling, but MongoDB or another document database is not required.

Final recommendation: Use PostgreSQL as the main database and optionally Redis for caching.

5.8 Search and Caching

Optional but useful

- Redis for caching and fast access to recent events, dashboard counters, or temporary session data
- PostgreSQL full text search initially for product search
- Elasticsearch or OpenSearch only if extra time remains, otherwise not necessary for a 22-day project

5.9 Big Data Tools

For a 22-day project, full-scale big data infrastructure is not required unless the dataset volume is very large.

Recommended approach

- Use pandas for initial development and feature engineering
- Use PostgreSQL for operational queries and reporting
- Add PySpark only if data volume is too large to process comfortably in memory

Big data tool recommendation

Primary recommendation: Do not make PySpark mandatory in the first version

Use PySpark only if

- The provided transaction dataset is very large
- Batch feature generation becomes slow in pandas
- You want to demonstrate large-scale data processing explicitly

Best practical choice

- Core system: PostgreSQL + Python + pandas
- Optional scale layer: PySpark for offline feature generation and training data preparation

6. Database Design Overview

6.1 Core Operational Tables

- users
- roles
- customer_profiles
- sales_representatives
- products
- product_categories
- orders
- order_items
- invoices
- payments
- wishlists
- wishlist_items
- recommendation_logs
- recommendation_clicks
- recommendation_conversions
- customer_events
- audit_logs

6.2 Analytics and Feature Tables

- customer_features
- product_features
- customer_item_interactions
- customer_cluster_assignments
- product_demand_summary
- daily_sales_aggregates
- rep_sales_aggregates
- recommendation_feature_store

6.3 Suggested Entity Relationships

- One user has one role
- One customer can have many orders
- One order can have many order items
- One order generates one invoice
- One customer can have many wishlist items
- One customer can receive many recommendations
- One product can appear in many orders, wishlists, and recommendations

7. Security and JWT Authentication Design

7.1 JWT Token Strategy

The system will use JWT based authentication with access tokens and refresh tokens.

Access token

- Short-lived token used for API access
- Sent in the Authorization header as Bearer token
- Contains user ID, role, and expiration metadata

Refresh token

- Longer-lived token used to obtain a new access token
- Stored securely and used during session renewal
- Helps maintain login state without requiring repeated credential entry

7.2 Authentication Flow

1. User submits login credentials
2. Backend validates user credentials against stored hashed password
3. Backend generates access token and refresh token
4. Access token is used for all protected API requests
5. Refresh token is used when the access token expires
6. Protected routes check token validity and role permissions

7.3 Role-Based Authorization

JWT payload will include user role, for example:

- admin
- salesperson
- customer

Protected endpoints will validate both token authenticity and role permissions.

Examples:

- Admin endpoints only accessible to admin users
- Customer profile edit endpoints restricted to authorized user or admin
- Sales recommendation endpoints accessible to assigned sales representatives and admins

7.4 Password Security

- Passwords will be hashed using bcrypt or passlib
- No plaintext password storage
- Secure login validation flow

7.5 Additional Security Considerations

- HTTPS in deployment environment
- Input validation on all APIs
- CORS configuration
- Rate limiting if time permits
- Audit logs for sensitive admin actions

8. Real-Time Architecture Design

8.1 Event-Driven Pipeline

The system will follow an event-driven architecture where key application activities generate Kafka events.

Events to publish

- user_logged_in
- product_viewed
- product_wishlisted
- cart_updated
- order_created
- payment_completed

- invoice_generated
- recommendation_served
- recommendation_clicked
- recommendation_converted
- customer_profile_updated

8.2 Kafka Topics

Suggested Kafka topics:

- user-events
- customer-events
- product-events
- order-events
- payment-events
- recommendation-events
- analytics-events

8.3 Kafka Consumers

Kafka consumers can process events for:

- live dashboard aggregation
- notification generation
- behavior logging
- recommendation refresh triggers
- analytics summary updates

8.4 WebSocket Integration

WebSockets will push relevant real-time updates to connected frontend clients.

Admin live updates

- new order placed
- payment received
- live revenue total changed
- top product trend updated

Sales representative live updates

- customer updated wishlist
- customer placed order

- recommendation status change
- account activity updates

Customer live updates

- payment confirmation
- invoice generated
- order status updates

9. Machine Learning Modules and Their Feature Mapping

The platform should use a clear set of machine learning modules, each tied to a specific business feature.

9.1 Module 1: Customer Segmentation and Clustering

Purpose: Group customers into meaningful segments based on behavior and profile attributes.

Likely algorithms:

- K-Means
- Hierarchical clustering
- Gaussian Mixture Model if needed

Input features:

- total spend
- purchase frequency
- average order value
- number of unique products purchased
- number of categories purchased
- recency
- wishlist activity
- customer size and type
- location or region

Used in features:

- customer profile enrichment
- cluster-level recommendation logic
- admin cluster analysis dashboard
- salesperson account strategy insights

9.2 Module 2: Collaborative Recommendation Engine

Purpose: Recommend products based on similar customer purchase patterns.

Likely approaches:

- user-based collaborative filtering
- item-based collaborative filtering
- matrix factorization if time allows

Input data:

- customer-item interaction matrix
- historical purchases
- quantities and frequency

Used in features:

- personalized product recommendations for customers
- personalized pitch suggestions for sales representatives
- similar customer behavior recommendation logic

9.3 Module 3: Association Rule Mining

Purpose: Find products that are commonly purchased together.

Likely algorithms:

- Apriori
- FP-Growth

Metrics:

- support
- confidence
- lift

Used in features:

- frequently bought together
- cross-sell suggestions in cart and checkout
- sales representative pitch suggestions
- admin analysis of co-purchase patterns

9.4 Module 4: Propensity or Purchase Likelihood Model

Purpose: Predict the likelihood that a customer will purchase a recommended product.

Likely algorithms:

- XGBoost
- LightGBM
- Logistic Regression for baseline

Input features:

- recency and frequency behavior
- product category match
- customer cluster
- historical purchases
- wishlist signals
- previous recommendation interactions

Used in features:

- ranking recommendations in priority order
- sales representative recommendation scoring
- customer side recommendation prioritization

9.5 Module 5: Reorder Prediction Module

Purpose: Predict which products or categories a customer is likely to need again soon.

Likely techniques:

- rule-based reorder interval logic initially
- time-based predictive model if time permits

Input features:

- historical order intervals
- category level reorder cycles
- quantity consumption patterns

Used in features:

- reorder reminders
- salesperson proactive outreach suggestions
- customer reorder shortcuts

9.6 Module 6: Demand Forecasting for Admin Analytics

Purpose: Forecast demand trends for products or product categories.

Likely algorithms:

- Prophet
- ARIMA
- XGBoost time series if needed

Input data:

- historical sales by day or week
- category demand trends
- product level trends

Used in features:

- admin planning dashboard
- top trend products
- demand trend analysis

9.7 Module 7: Recommendation Explanation Layer

Purpose: Generate reason tags for each recommendation to make outputs interpretable.

Approach:

- rules built on top of model outputs
- cluster membership explanation
- association rule evidence
- purchase pattern explanation

Used in features:

- customer recommendations with clear reasoning
- salesperson recommendation explanation
- admin review of recommendation drivers

10. ML Module to Feature Mapping Summary

Feature	ML Module Used
Customer clusters in profile	Customer Segmentation and Clustering
Personalized customer recommendations	Collaborative Recommendation Engine, Propensity Model

Sales representative pitch suggestions	Collaborative Recommendation Engine, Association Rules, Propensity Model
Frequently bought together	Association Rule Mining
Reorder alerts	Reorder Prediction Module
Product ranking for pitch order	Propensity or Purchase Likelihood Model
Admin demand insights	Demand Forecasting
Recommendation reason labels	Recommendation Explanation Layer

11. API Design Overview

The backend will expose REST APIs and WebSocket endpoints.

11.1 Key REST API Groups

- /auth
- /users
- /customers
- /products
- /orders
- /payments
- /invoices
- /wishlist
- /recommendations
- /analytics
- /admin

11.2 Example Endpoints

- POST /auth/login
- POST /auth/refresh
- GET /products
- GET /products/{id}
- GET /customers/{id}/profile
- GET /customers/{id}/recommendations
- POST /wishlist/add
- POST /orders/create
- POST /payments/checkout
- GET /invoices/{id}

- GET /analytics/live-summary
- GET /admin/top-products

11.3 WebSocket Channels

- /ws/admin/live-sales
- /ws/sales/{salesperson_id}
- /ws/customer/{customer_id}

12. Suggested Project Folder Structure

12.1 Backend Structure

- app/
    - main.py
    - api/
    - core/
    - models/
    - schemas/
    - services/
    - db/
    - auth/
    - kafka/
    - websocket/
    - ml/
    - utils/

12.2 Frontend Structure

- src/
    - pages/
    - components/
    - hooks/
    - services/
    - store/
    - routes/
    - layouts/
    - utils/

13. Development Environment and Free Tooling

Recommended free tools:

- PostgreSQL
- pgAdmin or DBeaver
- Kafka in Docker
- FastAPI
- React
- Stripe test mode
- Docker Compose for local setup
- GitHub for version control
- GitHub Projects or Trello for team task tracking
- Postman or Bruno for API testing

14. Deployment Considerations

For the capstone demo, deployment can be split as follows:

- Frontend on Vercel or Netlify
- Backend on Render or Railway
- PostgreSQL on Supabase or Neon free tier
- Kafka locally for demo or via lightweight containerized local setup

If Kafka cloud hosting is difficult in free tiers, demo can be done in local Docker Compose setup and recorded or presented live locally.

15. Proposed MVP and Stretch Scope

15.1 MVP Scope

- Multi-role login and JWT authentication
- Product catalog and product search
- Customer profile page
- Wishlist functionality
- Order creation and Stripe test checkout
- Invoice generation
- Recommendation engine with at least clustering, association rules, and collaborative recommendation
- Admin dashboard with core KPIs
- Kafka event publishing and one or two live dashboard views using WebSockets

15.2 Stretch Scope

- Reorder prediction
- Demand forecasting
- Recommendation explanation engine
- Enhanced live alerts
- Estimated insurance adjusted payable amount
- Redis caching

16. Testing Strategy

Backend testing

- unit tests for business logic
- API tests for auth, products, orders, recommendations
- integration tests for order to payment flow

Frontend testing

- component tests if time permits
- workflow testing for login, cart, checkout, dashboard use

ML testing

- offline evaluation metrics
- recommendation hit rate or precision at K
- cluster validation metrics such as silhouette score

Real-time testing

- Kafka event flow verification
- WebSocket update validation

17. Key Success Metrics

Business and technical success can be measured through:

- recommendation precision at K
- recommendation conversion rate
- wishlist to purchase conversion rate
- average basket size
- time to generate personalized recommendations
- dashboard live update latency
- order and payment completion rate

18. Risks and Mitigation

Risk 1: Scope too large for 22 days

Mitigation: Prioritize MVP features and mark others as stretch goals.

Risk 2: Kafka adds operational complexity

Mitigation: Use Docker Compose and keep Kafka usage limited to key events and dashboards.

Risk 3: Too many ML models to implement fully

Mitigation: Build 3 strong ML modules first, then add others if time remains.

Risk 4: Frontend complexity across three roles

Mitigation: Reuse shared components and layouts, then apply role-based route guards.

19. Recommended Final Technical Decisions

Core stack

- Frontend: React
- Backend: FastAPI
- ML: Python
- Event streaming: Kafka
- Real-time UI: WebSockets
- Payments: Stripe test mode
- Database: PostgreSQL
- Optional cache: Redis
- Optional big data tool: PySpark only if dataset scale requires it

Final database choice

Use PostgreSQL as the primary database

This is the best choice because the system is transaction heavy, relationship heavy, analytics friendly, and free.

20. 22-Day Full-Time Execution Plan

Day 1

- Finalize scope and feature prioritization
- Confirm roles, workflows, and MVP versus stretch goals
- Assign team ownership areas
- Create architecture draft and project board

Day 2

- Design database schema
- Define entities and relationships
- Finalize API contracts
- Set up GitHub repository and branching strategy

Day 3

- Initialize FastAPI backend structure
- Initialize React frontend structure
- Set up PostgreSQL and migrations
- Configure shared environment settings

Day 4

- Implement user model, role model, and auth tables
- Build login and registration endpoints
- Add password hashing and JWT token generation
- Create protected route middleware on frontend

Day 5

- Build product catalog backend endpoints
- Build product list and product detail pages in frontend
- Add search and basic filtering

Day 6

- Build customer profile database models and endpoints
- Create customer profile page in frontend
- Add purchase history and basic summary widgets

Day 7

- Build wishlist backend and frontend flows

- Add wishlist database tables and APIs
- Connect customer and sales representative views to wishlist data

Day 8

- Build cart and order data models
- Create order placement API
- Create checkout flow and order summary UI

Day 9

- Integrate Stripe test checkout
- Implement payment status update handling
- Design invoice model and invoice generation logic

Day 10

- Build invoice APIs and invoice frontend pages
- Add tax calculation logic
- Finish core customer order to payment workflow

Day 11

- Set up Kafka with Docker Compose
- Define event schema and Kafka topics
- Publish events from order, payment, and wishlist workflows

Day 12

- Build Kafka consumers for live sales aggregation
- Create backend live summary service
- Begin WebSocket integration in FastAPI

Day 13

- Connect WebSocket live updates to admin dashboard
- Build live sales admin dashboard components
- Validate end-to-end event flow from order to live UI update

Day 14

- Build data preprocessing scripts for ML
- Explore and clean customer, product, and sales datasets
- Engineer customer and product features

Day 15

- Implement customer clustering module
- Generate cluster assignments
- Surface cluster labels in customer profiles and admin insights

Day 16

- Implement association rule mining for frequently bought together
- Integrate results into recommendation service
- Expose recommendation API outputs

Day 17

- Implement collaborative recommendation logic
- Combine recommendation candidates from clustering and collaborative methods
- Create recommendation ranking strategy

Day 18

- Implement propensity scoring or ranking model
- Add explanation rules for recommendation reasons
- Integrate final ranked recommendations into customer and sales representative views

Day 19

- Build admin analytics pages for top products, revenue trends, and rep performance
- Add recommendation conversion and wishlist conversion analytics

Day 20

- Polish salesperson dashboard
- Add reorder or next likely purchase indicators
- Improve UX and navigation across all roles

Day 21

- Testing and bug fixing across backend, frontend, ML, and real-time modules
- Performance validation
- Demo data preparation
- Documentation writing

Day 22

- Final integration check
- End-to-end demo rehearsal
- Prepare presentation slides and architecture visuals
- Prepare final report and submission materials

21. Team Role Recommendation

If the team has multiple members, work can be divided as follows:

Backend lead

- FastAPI APIs
- auth
- order and invoice logic
- Stripe integration
- PostgreSQL integration

Frontend lead

- React pages and dashboards
- role-based routing
- WebSocket client integration
- user flows

ML lead

- feature engineering
- clustering
- recommendation modules
- model evaluation

Streaming and integration lead

- Kafka setup
- event producers and consumers
- WebSocket event relay

- live dashboard integration

If the team is smaller, these responsibilities can be merged.

22. Final Recommendation

This project should be positioned as a real-time intelligent sales and recommendation platform for medical-surgical products. The architecture should emphasize business usefulness first and technical depth second. PostgreSQL should be used as the primary free database. JWT based authentication should secure all protected APIs. Kafka and WebSockets should be used selectively but effectively to demonstrate real-time engineering capability. Machine learning should focus on customer clustering, personalized recommendations, association rule mining, ranking, and reorder intelligence.

The final system will demonstrate end-to-end product thinking, strong backend and frontend integration, applied machine learning, real-time event processing, analytics, and business alignment.